AI と一緒に JavaScript エンジンを作った話

今日話すこと

普段なにげなく書いている JavaScript。

```
console.log(1 + 2);
```

この 1 行を実行するために、エンジンの中で何が起きているか？

→ AI (Claude) と一緒に、JavaScript エンジンを丸ごと作ってみた。

デモ: Playground で見てみよう

https://yosuke-furukawa.github.io/jsmini/

デモ 1: `console.log(1 + 2)` の裏側 (1/2)

Playground に入力して各タブを見る:

Tokens タブ:

```
[console] [.] [log] [(] [1] [+] [2] [)] [;]
```

→ 文字列を「単語」に分割する

AST タブ:

```
CallExpression
  └── console.log
        └── BinaryExpression(1 + 2)
```

→ 単語の並びから「構造」を理解する

デモ 1: `console.log(1 + 2)` の裏側 (2/2)

Bytecode タブ:

```
LdaConst 1
LdaConst 2
Add
...
```

→ コンピュータが実行しやすい「命令列」に変換する

たった 1 行のコードのために、3 段階の変換が起きている。

デモ 2: 速度の違いを体感する

fibonacci(25) を実行して、モードを切り替える:

モード	時間	やっていること
Tree-Walking	450ms	木構造を素朴に辿る
Bytecode VM	90ms	命令列を順に実行
Wasm JIT	0.4ms	機械語にコンパイル

同じコードなのに 1000 倍速い。

→ なぜこんなに差が出るのか？

デモ 3: 最適化の before/after

IR Optimizations プリセットを選択:

```
// 最適化前: ループの中で毎回 x * 2 を計算
for (i = 0; i < n; i++) { sum += x * 2; }

// 最適化後: x * 2 をループの外に出す
t = x * 2;
for (i = 0; i < n; i++) { sum += t; }
```

コンパイラが自動でコードを書き換えて速くしている。

IR タブで実際に見える。

どうやって作ったか

5 ステップで段階的に

```
Source Code
  → ステップ 1: 文字列を単語に分割（Lexer）
    → ステップ 2: 構造を理解する（Parser）
      → ステップ 3: 素朴に実行する（Tree-Walking）
        → ステップ 4: 命令列にして速くする（Bytecode VM）
          → ステップ 5: 機械語にしてもっと速くする（JIT）
```

ステップ 1: 文字列を単語に分割する (Lexer)

```
"var x = 1 + 2;"
    ↓
[var] [x] [=] [1] [+] [2] [;]
```

人間が文章を「単語に区切る」のと同じ。

ステップ 2: 構造を理解する (Parser)

```
[var] [x] [=] [1] [+] [2] [;]
    ↓
VariableDeclaration
  └── x = Add(1, 2)
```

単語の並びから「これは変数宣言で、右辺は足し算」という木構造を作る。

ステップ 3: 素朴に実行する (Tree-Walking)

```
木を上から順に辿る:
  "変数宣言だな"
    → "右辺を計算しよう"
      → "1 + 2 = 3"
    → "x に 3 を入れる"
```

シンプルだけど遅い。木を毎回辿り直すのが無駄。

ステップ 4: 命令列にして速くする (Bytecode VM)

```
LdaConst 1    // 1 を積む
LdaConst 2    // 2 を積む
Add            // 足す → 3
StaLocal 0     // x に入れる
```

木を辿らず、命令を順に実行するだけ。**2〜5 倍速くなる。**

Chrome の V8 エンジンもこの方式。

ステップ 5: もっと速くする (JIT)

```
「この関数、100 回も呼ばれてる。毎回数値だな」
    → 数値専用の機械語にコンパイル!
    → 1000 倍速くなる
```

- 実行中に「どんなデータが来るか」を観察する (型フィードバック)
- 観察結果に基づいて専用のコードを生成する
- もし予想が外れたら元に戻す

AI との協業の実際

開発サイクル

毎フェーズ、同じ 5 ステップを繰り返した:

```
① RESEARCH  — まず調べる
② PLAN      — 方針を決める
③ TODO      — タスクに分解する
④ Implement — 1 つずつ実装 + テスト + コミット
⑤ LEARN     — 何を学んだかまとめる
```

① RESEARCH — まず調べる

```
自分:  「Promise を実装したい」

AI:    V8 / JSC / SpiderMonkey の実装を調査
       → RESEARCH-Promise.md にまとめる
```

- 他のエンジンがどう実装しているか
- ECMAScript の仕様で何が定義されているか
- **いきなりコードを書かない。地図を先に作る。**

② PLAN — 方針を決める

RESEARCH を踏まえて:

- 何を実装するか
- **何を実装しないか** ← これが大事
- どの順番でやるか

例: 「Promise.all / race はエンジン内部の対応不要。JS で書ける」→ 後回し

③ TODO — タスクに分解する

```
TODO-Phase23.md:
  □ 23-1: Promise オブジェクト
  □ 23-2: Microtask キュー
  □ 23-3: Promise.resolve / reject
  □ 23-4: async/await
  □ 23-5: テスト + Playground
```

各 TODO は **1 コミットで完了できるサイズ** にする。

④ Implement — 1 つずつ実装する

```
1 TODO につき:
  AI がコード書く → テスト書く → 全テスト通る → コミット
    ↓
  次の TODO へ
```

テストが通らないとコミットしない。

⑤ LEARN — 振り返る

```
LEARN-Phase23.md:
- microtask queue は JS 配列で実装。V8 は C++ deque
- .then() は settled でも microtask 経由
- 想定と違ったこと: ...
```

次のフェーズで同じ失敗をしないための記録。

LEARN — 振り返りで同じ失敗を繰り返さない

毎フェーズ終了後に **LEARN-PhaseXX.md** を書かせた。

```
LEARN-Phase21.md:
- LICM（ループ最適化）は Wasm 上では速度差が出なかった
  → V8 が同じ最適化を既にやっているから
- ただし LoadProperty（メモリアクセス）の LICM は 79 倍効いた
  → V8 の Liftoff はこれをやらないから
```

→ 次の Phase で「Wasm で差が出る/出ない」を判断できるようになった

LEARN がないとどうなるか

```
Phase 21: LICM 実装 → 「速くならない...なぜ？」→ 調査 → V8 がやってた
Phase 22: CSE 実装 → 「速くならない...なぜ？」→ 調査 → V8 がやってた（同じ失敗）
Phase 23: SR 実装  → 「速くならない...なぜ？」→ 調査 → V8 が（ry
```

LEARN があれば:

```
Phase 21: LICM → 速くならない → LEARN に記録: 「V8 がやるので差が出ない」
Phase 22: LEARN を読む → 「純粋算術は差が出ない。メモリアクセスなら効く」
          → LoadProperty の LICM に集中 → 79 倍
```

AI は前の会話を覚えていない。LEARN がその記憶の代わりになる。

AI のアウトプットをどう制御するか

AI の出力は「揺らぐ」

同じ指示でも、出力がブレる。

- 方向性が違う
- 余計な機能を足す
- 変数名を間違える (`fn.closure` を `fn.closureEnv` と書く)

「いい感じに作って」だと振れ幅が大きすぎる。

RESEARCH → TODO のサイクルで振れ幅を狭める

```
RESEARCH: 調べて認識を揃える → AI と人間が同じ地図を見る
TODO:     スコープを限定する → AI の 1 回の出力が小さくなる
テスト:   正しさを検証する   → 間違えても検出できる
```

範囲を限定し、範囲から漏れても修正できるようにする。

jsmini が上手くいった理由: test262

JavaScript には **test262** という公式テストスイート (1 万件以上) がある。

- AI が書いたコードが仕様通りか → **test262 で検証**
- AI の設計判断が妥当か → **V8 の実装と比較**
- 見落としたエッジケース → **test262 が検出**

「正解」が存在する領域だからこそ、AI の出力を検証・修正できた。

では「正解がない領域」ではどうするか？

段階的に振れ幅を狭めていく

```
1. 正解やお手本がないことは多い
     ↓
2. 作れるところから作ってみる
     ↓
3. それでもぶれるならハーネスを作る
     ↓
4. 開発の外側で早めに検証する
```

1. 正解やお手本がないことは多い

- 新規機能、社内ツール、前例のないもの
- AI に「いい感じに」では振れ幅が大きすぎる

→ まず認める。完璧な正解は最初からなくていい。

2. 作れるところから作ってみる

- 単体テストを先に書く — 「この入力にはこの出力」を 10 件でいい
- サンプルデータを用意する
- jsmini では: テスト 753 件を 1 TODO ごとに書いた

→ 小さな「正解」を部分的に作る。

3. それでもぶれるならハーネスを作る

- コーディングガイドライン — AI が従うべきルール
- E2E テスト — 全体の整合性を検出
- jsmini では: CLAUDE.md + test262

→ 個々のテストでは見つからない「全体の枠」を作る。

4. 開発の外側で早めに検証する

- QA に早めに繋げる
- 仕様確認会で PM やディレクターに触ってもらう
- jsmini では: Playground で自分が触って「おかしい」を見つけた

→ 作った人 (AI) と検証する人を分ける。

AI が得意だったこと

- **コード生成** — 「こういう関数が欲しい」→ すぐ書ける
- **テスト生成** — 正常系 + エッジケースを網羅的に
- **調査** — 「V8 はどうやってる？」→ 仕様書から要約
- **リファクタリング** — 安全に分割・統合

AI が苦手だったこと

- **方向性の判断** — 「今やるべき？やめとく？」は人間
- **微妙なバグの根本原因** — 「それっぽい修正」をするが根本が違うことがある
- **ベンチの解釈** — 数字が出ても「なぜ」は人間が考える
- **「やめるべき」判断** — 大きすぎる修正でも突き進もうとする

結果

何をしたか	効果
Tree-Walking (素朴な実行)	baseline
Bytecode VM を作った	**2〜5 倍速**
Wasm JIT を作った	**1000 倍速**
ループ途中から JIT 切り替え (OSR)	**215 倍速**

数字

- 構築: **24 Phase**、1 人 + AI
- テスト: **753 個**、全パス
- test262 (公式テスト) Promise: **49% 通過**
- コード: TypeScript のみ (C++ なし)
- ドキュメント: RESEARCH / PLAN / TODO / LEARN 合計 **120+ ファイル**

まとめ

AI があれば「作って理解する」ができる

- V8 のソース (C++ 30 万行) は読めなくても、同じ仕組みを TypeScript で作れた
- 「作る」と「読む」では理解の深さが違う
- JIT が速い理由、なぜ Deoptimization が必要か、等は作って初めてわかった

AI との協業で大事なこと

1. **小さく作る** — 大きな変更を一気にやらない
2. **テストで守る** — AI が間違えても検出できるように
3. **AI を疑う** — 出力を鵜呑みにしない。「本当？」が大事

正解がなくても大丈夫

1. 作れるところから正解を作る (単体テスト)
2. 全体の枠を作る (ガイドライン、E2E)
3. 外部の目で早めに検証する (QA、PM)

AI の振れ幅を、段階的に狭めていく。

試してみてください

https://yosuke-furukawa.github.io/jsmini/

ありがとうございました